

Tactic Cheat Sheet

<code>rfl</code>	beweist Aussagen der Form $a = a$. Kann (sehr) einfache Rechnungen durchführen.
<code>ring</code>	beweist Aussagen, die in allgemeinen kommutativen Ringen mit 1 gelten. Hat keine Kenntnis vom lokalen Kontext, kann also z.B. nicht die Definition von Variablen einsetzen.
<code>rw [h]</code>	Ersetzt im Ziel die linke Seite einer bestimmten Gleichung aus dem lokalen Kontext durch die rechte Seite.
<code>rw [←h]</code>	Ersetzt im Ziel die rechte Seite einer bestimmten Gleichung aus dem lokalen Kontext durch die linke Seite.
<code>nth_rewrite n [h]</code>	Führt die Operation <code>rw</code> nur für das n -te Vorkommen der linken Seite der entsprechenden Gleichung im Ziel durch.
<code>assumption</code>	Beendet einen Beweis, wenn das Ziel bereits im lokalen Kontext steht.
<code>exact [_]</code>	Beendet einen Beweis, wenn das Ziel der Variable in den eckigen Klammern entspricht. Dabei kann in den eckigen Klammern entweder eine Variable aus dem lokalen Kontext oder ein Lemma aus <code>mathlib</code> stehen.
<code>have</code>	Führt neue lokale Annahmen ein. Unterstützt <code>proof irrelevance</code> , d.h. vergisst die konkrete Definition des Objekts und merkt sich nur den Typen (dementsprechend nicht geeignet für die Definition von Funktionen etc.)
<code>intro</code>	Wenn das Ziel vom Typ $A \rightarrow B$ ist, ersetzt <code>intro h</code> das Ziel durch den Typ B und führt eine Variable $h : A$ ein. Wenn das Ziel vom Typ $\forall x, P\ x$ ist, führt <code>intro x</code> ein beliebiges x vom richtigen Typ ein und ersetzt das Ziel durch die Aussage, dass für dieses konkrete x $P\ x$ gilt.
<code>apply h</code>	Wenn $h : A \rightarrow B$ im lokalen Kontext ist und das Ziel B ist, ersetzt <code>apply h</code> das Ziel durch A .
<code>apply _ at _</code>	Wenn $h1 : A \rightarrow B$ und $h2 : A$ im lokalen Kontext sind, dann ändert <code>apply h1 at h2</code> die Annahme $h2$ in $h2 : B$.
<code>specialize</code>	Wenn $h1 : A \rightarrow B$ und $h2 : A$ im lokalen Kontext sind, dann ändert <code>specialize h1 h2</code> $h1$ zu $h1 : B$. Wenn $a : \alpha$ und $h : \forall x : \alpha, P\ x$ im lokalen Kontext sind, dann ändert <code>specialize h a</code> die Aussage h in $h : P\ a$.

<code>exfalso</code>	Wenn eine der Annahmen im lokalen Kontext den Typen <code>False</code> hat, beweist <code>exfalso</code> das Ziel, egal, was das Ziel ist (ex falso quod libet).
<code>contradiction</code>	Wenn der lokale Kontext sowohl einen Beweis von <code>A</code> als auch einen Beweis von der Negation von <code>A</code> enthält, beendet <code>contradiction</code> den Beweis.
<code>simp</code>	Vereinfacht das Ziel. Falls es als Zwischenschritt verwendet wird, sollte man <code>simp?</code> verwenden und dann den Vorschlag akzeptieren.
<code>constructor</code>	Teilt ein Ziel der Form $A \wedge B$ oder $A \leftrightarrow B$ in zwei Ziele auf.
<code>left / right</code>	Wenn das Ziel von der Form $A \vee B$ ist, legt <code>left</code> bzw. <code>right</code> eine der beiden Aussagen fest, die bewiesen wird.
<code>obtain ⟨h1, h2⟩</code>	Generiert zwei neue lokale Annahmen aus einer Annahme der Form $A \wedge B$ oder $A \leftrightarrow B$.
<code>obtain h1 h2</code>	Teilt einen Beweis, in dem eine Annahme von der Form $A \vee B$ ist, in zwei Fälle auf: einen mit <code>A</code> als Annahme und einen mit <code>B</code> als Annahme.
<code>all_goals</code>	In einer Situation, in der es mehrere offene Ziele gibt, führt <code>all_goals</code> dazu, dass alle folgenden Taktiken auf allen existenten Zielen ausgeführt werden (nicht nur auf dem ersten).
<code>obtain ⟨x, hx⟩</code>	Generiert ein $x : \alpha$ und eine Annahme $hx : P\ x$ aus einer Annahme der Form $\exists x : \alpha, P\ x$.
<code>use x</code>	Beweist Existenzaussagen durch angeben eines <code>x</code> mit dem richtigen Typ, das die Existenzaussage erfüllt.
<code>unfold</code>	Ersetzt vorher definierte Begriffe durch ihre Definition / Konstruktion (im Ziel oder lokalen Kontext).
<code>by_contra h</code>	Beginnt einen Widerspruchsbeweis: Wenn <code>A</code> das Ziel ist, ersetzt <code>by_contra h</code> das Ziel durch <code>False</code> und führt $\neg A$ als neue Assumption im lokalen Kontext ein.
<code>by_cases h : _</code>	Beginnt eine Fallunterscheidung, bei der im einen Fall die Aussage von <code>h</code> und im zweiten Fall die Negation von <code>h</code> vorausgesetzt wird.
<code>push Not</code>	Schiebt Negationen in die scope von Quantoren (und dreht dabei Quantoren um).
<code>rw?</code>	Sucht in mathlib nach einem möglichen Lemma zum Rewriten.
<code>exact?</code>	Sucht in mathlib ein Lemma, das das Ziel direkt löst.